WAYGROUND formerly Quizizz Worksheets

Name

FTW_Databricks Data Engineer Associate V3.3

Class

Total questions: 90

Date

Worksheet time: 46mins

1. A data engineer maintains a Declarative Automation Bundle (formerly Databricks Asset Bundle), which deploys a dashboard. The databricks. ym includes: bundle: name: workspace_assets variables: dataset_catalog: default: catalog_dev targets: dev: variables: dataset_catalog: catalog *dev prod: variables: dataset*catalog: catalog_prod
   Which set of actions will enable the dashboard to be deployed in prod using catalog_prod without changing the resource definition?

   a) Rundatabricks bundle deploy --profile prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically.

   b) Run databricks bundle deploy --target prod so the deployment uses targets .prod and overrides dataset_catalog to catalog_prod

   c) Rundatabricks bundle execute --profile prod so the CLI selects targets.prod and applies dataset_catalog: catalog_prod automatically.

   d) Run databricks bundle deploy --var dataset_catalog=catalog_prod so the CLI selects targets .prod and applies dataset_catalog: catalog_prod automatically.

2. A data engineer observes that an ETL job in Databricks has increased its execution time from 5 minutes (baseline) to 12 minutes over the past week. Using the Lakeflow Jobs run history view, the engineer needs to identify whether the slowdown is consistent or intermittent and determine the root cause.
   Which action should the engineer take to analyze execution time trends against the baseline?

   a) Compare cluster metrics in Lakeflow Jobs run history to spot resource spikes causing delays.

   b) Archive old job runs and resets the baseline counter to ensure accurate comparison of current performance.

   c) Rerun the job immediately to establish a new baseline since historical data becomes irrelevant after seven days.

   d) Increase cluster size automatically whenever execution time exceeds the previous run's duration.

3. A data engineering team uses Declarative Automation Bundles (formerly Databricks Asset Bundles) to deploy the same codebase across dev, test, and prod environments. The team wants environment-specific behavior to be applied only through bundle configuration, without modifying notebooks, job definitions, or deployment logic during promotion.
   Which approach applies environment-specific configuration while meeting this requirement?

   a) Use separate Git branches per environment so each branch can contain environment specific configuration values.
   b) Define bundle variables and reference them from bundle resources, then override variable values per environment using bundle targets.
   c) Parameterize environment-specific values inside notebooks and set those parameters when each job run is triggered.
   d) Deploy the same bundle once and edit job settings in the workspace UI for each environment after deployment.

4. A data engineer configures a Databricks Lakeflow Job for daily customer data processing: + Entry task: Single notebook loads raw data » Parallel tasks: o SQL query task for cleansing o Notebook task for feature engineering o Pipeline task for model updates » Exit task: Dashboard refresh (must run after all parallel tasks complete) + Requirement: Implement this dependency pattern using a DAG-based task graph
   Which task configuration ensures all parallel tasks complete before the dashboard refresh task runs?

   a) Create separate Lakeflow Jobs for each parallel task and trigger them sequentially using external orchestration.
   b) Add the dashboard task as dependent on all three parallel tasks using fan-in control flow.
   c) Configure the parallel tasks with sequential dependencies so each task waits for the previous one to finish.
   d) Set the dashboard task to depend only on the SQL query task; other tasks run independently.

5. A data engineer is designing a cost-optimized, event-driven pipeline. They configure a Lakeflow Job with a File Arrival trigger to watch an S3 bucket. The job runs a notebook that uses Auto Loader with trigger (availableNow=True) to ingest data into a Bronze table.
   What is the technical relationship between the File Arrival trigger and Auto Loader in this integration pattern?

   a) Auto Loader must be configured in File Notification mode to work with a Lakeflow Job File Arrival trigger.

   b) The File Arrival trigger starts the job run, while Auto Loader uses its internal checkpoint to independently identify and process only the new files that have arrived since the last successful commit.

   c) The File Arrival trigger automatically passes the specific file path of the new arrival to Auto Loader, allowing the engineer to skip the source path configuration in the code.

   d) Using a File Arrival trigger requires the engineer to disable Auto Loader's checkpointing to prevent the job from processing the same file multiple times.

6. A data engineer has manually created several Databricks jobs and dashboards using the workspace UI. The team now wants to manage these resources as code using Declarative Automation Bundles (formerly Databricks Asset Bundles), store the configuration in a Git repository, and deploy changes through CI/CD.
   Which approach converts the existing resources into a bundle project?

   a) Run databricks bundle init to create a new bundle project, then use databricks bundle deploy pointed at the workspace to automatically detect and import all existing resources into the bundle configuration.

   b) Use the Databricks REST API to export each job's JSON definition, save the JSON files into a Git repository, and configure CI/CD to import them using the REST API on each deployment.

   c) Run databricks bundle generate for each existing resource, use --bind to link the generated configuration to the existing workspace resources, and commit the bundle project to Git for CI/CD deployment.

   d) Clone an empty Git repository into a Git Folder, recreate each job and dashboard configuration in YAML by referencing the settings using workspace UI, then commit and deploy with databricks bundle deploy.

7. A data engineer is troubleshooting a Lakeflow Spark Declarative Pipelines (SDP) where several rows are dropped by expectations. The product owner requests details on which expectation failed most frequently during the last run and the corresponding affected table. The engineer wants to use Databricks' built-in observability data to answer this without adding new logging code.
How should the data engineer troubleshoot SDP to verify the expectation failure?

   a) Export notebook run results to HTML and search for the expectations summary in the rendered output to compile the metrics.
   b) Query the Lakeflow Spark Declarative Pipelines event log to retrieve expectation metrics and failure details for the pipeline's most recent update.
   c) Use the Spark UI's SQL tab to identify failed rows and expectation names from physical query plans.
   d) Increase the cluster's log level to DEBUG and parse the driver's Log4j logs for expectation metrics for the last run.

8. A data engineer is investigating a Lakehouse ETL job on Databricks that has suddenly become four times slower. They need to: - Identify the specific Spark stages that regressed. + Drill down to per-task metrics such as shuffle read/write size, spill, and GC time. » Correlate these with the executor resource usage during that run.
Which Databricks-native capability is the starting point for this detailed job-level root-cause analysis?

   a) Cluster Metrics (Metrics tab on the compute) to view historical Spark and hardware metrics, without using the Spark UI.
   b) The Jobs run details page timeline and graph views, without opening the Spark UI, to analyze all stage and task-level Spark metrics.
   c) Ganglia metrics for the cluster, which provide full visibility into Spark stages, tasks, and shuffle metrics for the job run.
   d) Spark UI opened from the job run's compute, using the Jobs and Stages tabs to inspect task metrics.

9. A Lakeflow Jobs workflow fails at the last task because of an error in a notebook task. This workflow is scheduled to run daily. The data engineer fixes the notebook and wants to rerun the pipeline. This workflow is very compute-intensive and expensive to run.
Which action should the data engineer take in order to minimize downtime and compute cost?

   a) Restart the cluster
   b) Re-run the entire workflow
   c) Switch to another cluster
   d) Repair run

10. After a failed multi-task job run, a data engineer changes the cluster size and updates a notebook path for one task. The data engineer wants to re-run just the failed/skipped tasks with the updated settings and keep the run lineage intact.
Which strategy should the data engineer use to preserve the run history?

a) Use Jobs REST API repairRun to re-run failed tasks; the repair uses the current job and task settings and appears in the run history.
b) Increase the task retry count and click 'Retry' on the failed task to automatically apply new cluster settings within the same run.
c) Export the job as a Databricks Asset Bundle, redeploy it, and re-run only the modified tasks from the bundle.
d) Use 'Run now' to start a new run, so that the updated settings apply; the new run replaces the prior run history.

11. A team is designing their Data bricks Asset Bundle for deployment. They want to ensure their bundle includes all necessary resources, targeted environment configurations, and deployment settings in a single version-controlled file.
What is required as the main configuration file at the root of their project?

a) A Python script called configure_bundle.py that dynamically creates resources at runtime
b) A databricks.yml YAML file that contains the bundle's name, targets, includes resources, and variables
c) A directory with multiple TXT files, each describing a single resource for the bundle
d) A JSON file named bundle_config.json that stores all resource details and environment variables

12. A data engineer is standardizing repository layouts for multiple teams adopting Databricks Asset Bundles. The engineer wants to ensure every project has a single authoritative configuration file at the repository root that defines the bundle name, targets, workspace settings, permissions, and resource mappings (for jobs and pipelines).
What strategy should the data engineer use to meet the goal?

a) Place multiple databricks.yml files under each subfolder (for example, jobs/pipelines/, workspace/) and merge them at deploy time using the include mapping.
b) Place exactly one databricks.yml at the repository root; it is the main configuration file and may reference additional configuration files via the include mapping.
c) Place a databricks. ym] in a databricks/hidden folder at the repository root; only hidden locations are valid for bundle configs.
d) Place a databricks.yaml at the repository root and optional databricks.ym1 in subfolders; the CLI prefers .yam1 over . ym1 when both exist.

13. A data engineer is migrating pipeline tasks to reduce operational toil. The workspace used Unity Catalog and is in a region that supports serverless. The engineer wants Databricks to auto-select instance types, manage scaling, apply Photon, and handle runtime upgrades automatically for job runs.
How should the data engineer meet this requirement while adhering to Databricks constraints?

a) Run the job on a serverless compute for workflows configuration, ensuring Unity Catalog is enabled and regional support is available.
b) Use a Pro SQL warehouse and schedule Python notebook tasks to execute as pipeline steps.
c) Create a job with a single-task job cluster and manually set instance families and min/max workers.
d) Use an all-purpose cluster with cluster policies to enforce standard sizes and enable autoscaling.

14. A Databricks single-task workflow fails due to an error in a notebook. The data engineer fixes the mistake in the notebook.
What should the data engineer do to rerun the workflow?

a) Repair the task
b) Repair the run
c) Restart the Cluster
d) Swap the cluster

15. Which method should a Data Engineer apply to ensure Workflows are being triggered on schedule?

a) Scheduled Workflows require an always-running cluster, which is more expensive but reduces processing latency.
b) Scheduled Workflows process data as it arrives at configured sources.
c) Scheduled Workflows can reduce resource consumption and expense since the cluster runs only long enough to execute the pipeline.
d) Scheduled Workflows run continuously until manually stopped.

16. A data engineer and data analyst are working together on a data pipeline. The data engineer is working on the raw, bronze, and silver layers of the pipeline using Python, and the data analyst is working on the gold layer of the pipeline using SQL. The raw source of the pipeline is a streaming input. They now want to migrate their pipeline to use Delta Live Tables.
Which change will need to be made to the pipeline when migrating to Delta Live Tables?

a) The pipeline can have different notebook sources in SQL & Python.
b) The pipeline will need to be written entirely in SQL.
c) The pipeline will need to use a batch source in place of a streaming source.
d) The pipeline will need to be written entirely in Python.

17. Differentiate between all-purpose clusters and jobs clusters. A data engineering team has created a python notebook to load data from cloud storage, this job has been tested and now needs to be scheduled in production.
Which would be the best cluster to be used in this case?

a) All purpose cluster
b) Any Unity Catalog-enabled cluster
c) Jobs Cluster
d) Serverless SQL warehouse

18. A Delta Live Table pipeline includes two datasets defined using STREAMING LIVE TABLE. Three datasets are defined against Delta Lake table sources using LIVE TABLE. The table is configured to run in Production mode using the Continuous Pipeline Mode.
What is the expected outcome after clicking Start to update the pipeline assuming previously unprocessed data exists and all definitions are valid?

a) All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will persist to allow for additional testing.
b) All datasets will be updated once and the pipeline will shut down. The compute resources will persist to allow for additional testing.
c) All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped.
d) All datasets will be updated once and the pipeline will shut down. The compute resources will be terminated.

19. A data analyst has a series of queries in a SQL program. The data analyst wants this program to run every day. They only want the final query in the program to run on Sundays. They ask for help from the data engineering team to complete this task.
Which of the following approaches could be used by the data engineering team to complete this task?

a) They could submit a feature request with Databricks to add this functionality.

b) They could wrap the queries using PySpark and use Python's control flow system to determine when to run the final query.

c) They could only run the entire program on Sundays.

d) They could automatically restrict access to the source table in the final query so that it is only accessible on Sundays.

e) They could redesign the data model to separate the data used in the final query into a new table.

20. A pipeline uses COPY INTO to ingest CSV files from cloud object storage into a Unity Catalog Delta table. Some files are occasionally re-uploaded with corrections using the same filename. The engineer needs the corrected data ingested as soon as it occurs.
What should the engineer do?

a) Reload all files from the source path on each run

b) Load corrections as new files and apply updates to the target table

c) Recreate the target table before each run

d) Overwrite the target Delta table on each run

21. A data engineer needs to ingest JSON change data from Salesforce into Unity Catalog-governed Delta tables with a low-code, fully managed experience.
Which Databricks capability should the data engineer use?

a) A Lakeflow Connect managed Salesforce connector that writes into Unity Catalog Delta tables

b) Use the Salesforce REST API in a PySpark notebook to query changed records and write them to Delta tables

c) Auto Loader reading JSON files from an S3 bucket exported by Salesforce

d) Auto Loader with a file arrival trigger on DBFS

22. A data engineer converts an external table to a managed table. A structured streaming job that reads from this table continues running during the conversion. After the conversion completes, the streaming job stops processing new records.
How should the data engineer resolve this?

a) Grant the streaming job additional permissions on the new managed storage location
b) Restart the streaming job, so it picks up the new managed table location
c) Run REFRESH TABLE on the converted table to update the streaming checkpoint
d) Delete the streaming checkpoint directory and let the job reprocess from the beginning

23. A data engineer ingests semi-structured JSON logs into a Delta table using Auto Loader with schema evolution enabled. A new string field userAgent appears in the JSON.
What happens to the new userAgent field?

a) All existing data is backfilled with a default value for userAgent and made non nullable.
b) The new userAgent field is added as a new nullable column in the Delta table, populated for new data.
c) Ingestion fails with a schema-mismatch error until the table is altered.
d) The new userAgent field is silently ignored, and its data is dropped for all new files.

24. A data engineering team needs to integrate two data sources into Databricks: + Clickstream Events: Ingesting 5,000 events per second from an Apache Kafka topic. » Customer Master Data: Syncing only changed records every 4 hours from a Snowflake database. The solution must meet two key requirements: + Process the streaming clickstream data with a latency of under 30 seconds. » Prevent the reprocessing of customer master data that has not changed.
Which data ingestion approach meets these requirements?

a) Use Structured Streaming for Kafka and a Lakeflow Connect standard connector with full refresh for Snowflake.
b) Use Structured Streaming for Kafka and Lakeflow Connect managed connector with incremental mode for Snowflake.
c) Use spark. readStream() with Kafka and query Snowflake hourly with a time-based filter.
d) Use a non-incremental Snowflake connector with polling and fetch all data every 4 hours with MERGE operations.

25. A data engineering team needs to incrementally ingest customer transactions from a SaaS app into Data Intelligence Platform with: + Built-in change data capture (including updates and deletes) » Automatic schema evolution - Serverless execution with retries and minimal maintenance » OAuth support and basic monitoring
Which solution meets all of these requirements?

a) Use a Lakeflow Connect managed connector for the SaaS app with built-in CDC, schema evolution, OAuth, and serverless compute with retries, then process downstream with Lakeflow Spark Declarative Pipelines.

b) Use Auto Loader on exported Saa$ files and an SDP that calls the SaaS APIs, handling CDC, schema changes, OAuth, and retries in your own code.

c) Schedule batch jobs per region that call SaaS REST APIs and write directly to silver tables with custom CDC and schema evolution, and use system tables for ad hoc basic monitoring.

d) Use a Lakeflow Connect standard connector, implement MERGE-based CDC, schema evolution, and retries in workflows, and set up a dedicated monitoring dashboard.

26. A data engineering team needs to ingest historical CSV files from a cloud storage location that already contains 50,000 existing files. They also expect new files to arrive continuously going forward. The team wants to use Auto Loader to incrementally process both existing files and new arrivals efficiently.
Which Auto Loader mode should the team configure for this use case?

a) Use file notification mode because it scales efficiently for both large numbers of existing files and continuous new arrivals by using cloud storage events rather than repeatedly listing directories.

b) Use directory listing mode because it provides better performance for the initial processing of 50,000 existing files through optimized incremental listing capabilities.

c) Use directory listing mode initially to process the 50,000 existing files, then reconfigure the stream to use file notification mode for ongoing ingestion of new arrivals.

d) Use file notification mode only after all existing files have been processed, as it cannot efficiently handle large volumes of pre-existing files.

27. An organization is building a data lakehouse and needs to ingest data from multiple sources into Unity Catalog-managed tables: + Source 1: Salesforce (50+ objects, frequent schema changes, OAuth authentication required) » Source 2: On-premises SQL server database (100+ tables, CDC enabled, requires network connectivity) + Source 3: Daily JSON files landing in Azure Data Lake Storage Gen2 The organization wants all ingested data to be governed by Unity Catalog, to minimize data engineering effort for schema changes, and to use serverless wherever possible to reduce operational overhead.
Which ingestion strategy meets these requirements?

a) Use partner connectors (Fivetran) for all three sources via Partner Connect, as they provide the most comprehensive integration with Unity Catalog and automated schema evolution.

b) Use Lakeflow Connect managed connectors for all three sources, as they provide consistent serverless execution and Unity Catalog integration across all data source types.

c) Use Lakeflow Connect managed connectors for Salesforce and SQL Server, and Auto Loader for ADLS files, all within Lakeflow Spark Declarative Pipelines.

d) Use Auto Loader for all three sources with custom Python code to handle Salesforce API calls and SQL server CDC extraction, then write to Unity Catalog tables.

28. A team is using a Databricks workspace and needs to continuously ingest JSON files as they arrive.
Which code snippet shows a valid Auto Loader source configuration?

a)
```
spark.readStream \
    .format("json") \
    .option("cloudFiles.format", "json") \
    .load("<path>")
```

b)
```
spark.readStream \
    .format("cloudFiles") \
    .load("<path>")
```

c)
```
spark.readStream \
    .format("cloudFiles") \
    .option("format", "json") \
    .load("<path>")
```

d)
```
spark.readStream \
    .format("cloudFiles") \
    .option("cloudFiles.format", "json") \
    .load("<path>")
```

e)
```
spark.readStream \
    .option("cloudFiles.format", "json") \
    .load("<path>")
```

29. When using MERGE INTO in Delta Lake on Databricks, which clause allows you to atomically delete target rows that no longer match any source rows?

a) WHEN MATCHED BY SOURCE THEN UPDATE SET NULL

b) WHEN MATCHED AND THEN DELETE

c) WHEN NOT MATCHED BY SOURCE THEN DELETE

d) WHEN NOT MATCHED THEN DELETE

30. A data engineer streams customer orders into a Kafka topic (orders_topic) and is currently writing the ingestion script of Lakeflow Spark Declarative Pipelines (SDP). The data engineer needs to ingest the data from Kafka brokers to SDP using Databricks.
What is the correct code for ingesting the data?

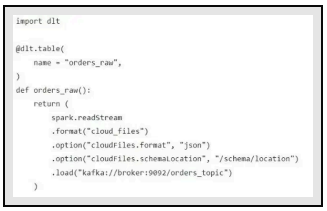
a)



```
import dlt

@dlt.table(
    name = "orders_raw",
)
def orders_raw():
    return (
        spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "broker:9092")
        .option("subscribe", "orders_topic")
        .option("startingOffsets", "earliest")
        .load()
    )
```

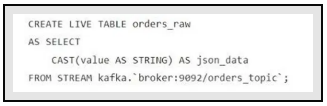
b)



```
import dlt

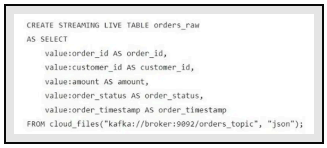
@dlt.table(
    name = "orders_raw",
)
def orders_raw():
    return (
        spark.readStream
        .format("cloud_files")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", "/schema/location")
        .load("kafka://broker:9092/orders_topic")
    )
```

c)



```
CREATE LIVE TABLE orders_raw
AS SELECT
    CAST(value AS STRING) AS json_data
FROM STREAM kafka.`broker:9092/orders_topic`;
```

d)



```
CREATE STREAMING LIVE TABLE orders_raw
AS SELECT
    value:order_id AS order_id,
    value:customer_id AS customer_id,
    value:amount AS amount,
    value:order_status AS order_status,
    value:order_timestamp AS order_timestamp
FROM cloud_files("kafka://broker:9092/orders_topic", "json");
```

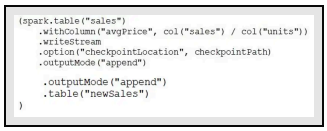
31. Which Structured Streaming query is performing a hop from a Silver table to a Gold table?

a)



```
(spark.readStream.load(rawSalesLocation)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

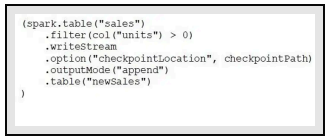
b)



```
(spark.table("sales")
    .withColumn("avgPrice", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")

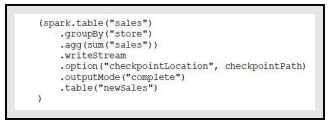
    .outputMode("append")
    .table("newSales")
)
```

c)



```
(spark.table("sales")
    .filter(col("units") > 0)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

d)



```
(spark.table("sales")
    .groupBy("store")
    .agg(sum("sales"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .table("newSales")
)
```

32.

```
(spark.readStream
      .table("sales")
      .withColumn("avg_price", col("sales") / col("units"))
      .writeStream
      .option("checkpointLocation", checkpointPath)
      .outputMode("complete")
      .______
      .table("new_sales")
)
```

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table. The code block used by the data engineer is below:

The data engineer only wants the query to process all of the available data in as many batches as required. Which line of code should the data engineer use to fill in the blank?

a) trigger (availableNow=True)

b) trigger (processingTime="once")

c) trigger (continuous="once")

d) trigger (once=True)

33. Which query is performing a streaming hop from raw data to a Bronze table?

a)

```
(spark.table("sales")
    .groupBy("store")
    .agg(sum("sales"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .table("newSales")
)
```

b)

```
(spark.read.load(rawSalesLocation)
    .write
    .mode("append")
    .table("newSales")
)
```

c)

```
(spark.table("sales")
    .withColumn("avgPrice", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

d)

```
(spark.readStream.load(rawSalesLocation)
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("append")
    .table("newSales")
)
```

34. What is used by Spark to record the offset range of the data being processed in each trigger in order for Structured Streaming to reliably track the exact progress of the processing so that it can handle any kind of failure by restarting and/or reprocessing?

a) Checkpointing and Write-ahead Logs

b) Replayable Sources and Idempotent Sinks

c) Write-ahead Logs and Idempotent Sinks

d) Checkpointing and Idempotent Sinks

35.

```
(spark.table("sales")
    .withColumn("avg_price", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")
    .______
    .table("new_sales")
)
```

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table. The code block used by the data engineer is below:

Which line of code should the data engineer use to fill in the blank if the data engineer only wants the query to execute a micro-batch to process data every 5 seconds?

a) trigger("5 seconds")

b) trigger (continuous="5 seconds")

c) trigger(once="5 seconds")

d) trigger (processingTime="5 seconds")

36.

```
(spark.table("sales")
    .withColumn("avg_price", col("sales") / col("units"))
    .writeStream
    .option("checkpointLocation", checkpointPath)
    .outputMode("complete")

                .outputMode("complete")
                .______
                .table("new_sales")
            )
```

A data engineer has configured a Structured Streaming job to read from a table, manipulate the data, and then perform a streaming write into a new table. The code block used by the data engineer is below:
If the data engineer only wants the query to execute a micro-batch to process data every 5 seconds, which of the following lines of code should the data engineer use to fill in the blank?

a) trigger("5 seconds")

b) trigger ()

c) trigger(once="5 seconds")

d) trigger (processingTime="5 seconds")

e) trigger(continuous="5 seconds")

37. Which Databricks SQL predicate correctly performs a null-safe equality comparison so that rows are matched when both sides are NULL or when both are equal non-NULL values?

a) WHERE customer_id <= > :cid

b) WHERE customer_id = :cid

c) WHERE COALESCE (customer_id, '') = COALESCE(:cid, '')

d) WHERE customer_id <> :cid OR customer_id IS NULL

38. A data engineer has two DataFrames: orders_df (all orders) and customers_df (only active customers). The data engineer needs to filter out orders placed by inactive customers.
Which operation should the data engineer use?

a) customers_df left join with orders_df
b) orders df and customers_df inner join
c) orders_df and customers_df cross join
d) orders_df left join with customers _df

39. A data engineer is designing a Gold layer table for a BI team that needs to query complex aggregations. The requirements state that the data should be precomputed to ensure low latency for dashboard users, but the underlying data changes only once every few hours.
Which object type should be implemented in Unity Catalog to meet these requirements?

a) Streaming table
b) External table
c) Materialized view
d) View

40. A data engineer investigates a slow stage in the Databricks Spark UI. The engineer scrolls down to the Summary Metrics section for task durations, and they observe for that stage: + 25th percentile duration: 40 seconds » Median duration: 45 seconds + 75th percentile duration: 50 seconds » Max duration: 80 seconds
What is the interpretation of these metrics?

a) There is a GC issue
b) The stage likely suffers from data skew.
c) The stage is healthy.
d) The cluster is under-provisioned.

41. A data engineer runs df. toPandas () on a wide DataFrame containing 50 million rows. The notebook cell fails with a java. 1lang.OutOfMemoryError on the driver.
Which memory configuration is directly responsible for this failure?

a) spark.memory . fraction is too low, limiting the storage memory available for caching
b) spark.executor.memory is too low to process the DataFrame transformations
c) spark.sql.shuffle.partitions is set too high, causing excessive memory consumption during shuffles
d) spark.driver.memory is too low to hold the collected result set

42. A data engineer is joining a very large events_df (billions of rows) with a small country *lookup*df (fewer than 10,000 rows) on the country _code column. The join is slow due to data shuffling.
How should the data engineer improve performance?

a) Cache both DataFrames, then run the same join
b) Persist both DataFrames, then run the same join
c) broadcast events_d£ when performing the join
d) broadcast country *lookup*df when performing the join

43. A data engineer is running a Spark SQL job on Databricks and notices that joins between a small lookup table and a large fact table are slow. The data engineer wants Spark to automatically send the small lookup table to all executors to speed up the join.
Which configuration parameter should the data engineer adjust?

a) spark.sql.shuffle.partitions
b) spark.executor.memory
c) spark.sql.autoBroadcastJoinThreshold
d) spark.default.parallelism

44. A data engineer needs to find all customers who have never placed an order. The customers table contains customer_id, customer_name, and other customer attributes. The orders table contains order_id, customer_id, and other order attributes. Both tables share customer_id as the join key. The result must include only customer columns with no order-related columns in the output.
Which expression exactly achieves this requirement?

a) Inner join, then filter for NULL order_id
b) Left anti-join from customers to orders
c) Left outer join from orders to customers
d) Full outer join, then filter where order_id IS NULL

45. A data engineer is tasked with cleaning a bronze table. The requirement is to eliminate rows where either the customer_email or the customer_phone field is null. This data cleaning must be performed in a single operation using a single method call.
Which PySpark approach supports filtering multiple columns for nulls in one call?

a) df. dropna(subset=['customer_email', 'customer_phone'])
b) df.where(' customer_email IS NOT NULL") .where('customer_phone IS NOT NULL")
c) df.na.drop (how='all')
d) df.filter(col(' customer_email').isNotNull() & col('customer_phone') .isNotNul11 ())

46. A data engineer is cleaning a bronze table that receives the same customer records from multiple source systems. Duplicate rows have the same customer_id and email but different ingestion_timestamp values. The silver table should contain only one record per unique combination of customer_id and email.
Which PySpark operation correctly deduplicates based on the business keys?

a) df.dropDuplicates(['customer_id', 'email'])

b) df.groupBy (' customer_id', 'email') .agg(max('ingestion_timestamp') .alias('latest_ts'))

c) df.select (' customer_id', 'email') .distinct ()

d) df.distinct ()

47. A data engineer has configured a Lakeflow Job that runs daily to ingest customer transaction data from a legacy relational database. The extracted data must be written directly to a Unity Catalog table and be immediately queryable via SQL. The team needs to ensure full data lineage is preserved.
Which action enables this data ingestion with direct landing in Unity Catalog, preserved lineage, and immediate SQL query ability?

a) Stage data in cloud storage via REST API, then load with a separate job

b) Use spark.read.format ("jdbc") and write with saveAsTable ("catalog.schema. table")

c) Extract to Pandas DataFrame, convert to Spark, then save () to external path

d) Use ODBC pooling to extract all records in one batch and write to /dbfs/

48. A data engineering team is designing the Gold layer in their Unity Catalog-governed lakehouse for downstream BI and analytics users. The team wants to expose business-ready metrics with fast query performance and consistent definitions, while keeping the transformation logic in Spark notebooks.
Which type of Gold layer object meets this requirement?

a) A materialized view that precomputes aggregations from Silver tables on a schedule and is queried by BI tools for faster, consistent analytics.

b) A streaming table that continuously ingests raw events from source systems and exposes them directly to BI users.

c) A view that directly references raw Bronze tables so BI tools can define their own joins and aggregations.

d) A collection of CSV files stored in a cloud storage object. BI users will parse the files to query the gold layer data.

49.

```
event_id STRING,
device STRUCT<id: STRING, model: STRING, location: STRUCT<latitude: DOUBLE, longitude: DOUBLE>>,
event_ts TIMESTAMP
```

A data engineer has a PySpark DataFrame events_df which contains a nested device struct, which itself includes a nested location struct, as shown:
The goal is to flatten the nested fields into root-level columns while keeping the event identifiers and timestamp. Which PySpark expression achieves this?

a)

```
events_df.select("event_id", "event_ts", "device.id", "device.model", "device.location.latitude",
"device.location.longitude")
```

b)

```
events_df.withColumn("device_id",    events_df["device.id"]).withColumn("device_model",
events_df["device.model"]).withColumn("location", events_df["device.location"])
```

c) events df.select("event_id", "event_ts", "device.*")

50. A data engineer is troubleshooting two different pipeline failures: + Pipeline A fails with a java. lang.OutOfMemoryError is immediately thrown after the command display (d£.collect()) is called on a 100GB dataset. » Pipeline B fails during a wide transformation (a join of two large tables) with an ExecutorLostFailure error message, indicating executor memory exhaustion during shuffle. Which action should the data engineer take to fix these two issues?

a) For Pipeline A, switch to a Storage Optimized cluster; for Pipeline B, use a Compute Optimized cluster.

b) For Pipeline A, increase the spark.driver.memory; for Pipeline B, increase the spark.sql.shuffle.partitions or executor memory.

c) For Pipeline A, increase the number of executors; for Pipeline B, increase the driver memory.

d) For both pipelines, enable Adaptive Query Execution (AQE) as it automatically manages memory allocation for both the driver and executors.

51. A data engineer is transforming a bronze table containing API response data into a silver table. The bronze table has a user_profile column of type STRING that contains JSON data. An example value is: The silver table needs to make this data easily queryable for analytics without requiring JSON parsing in every downstream query.
Which approach standardizes this column for the silver table?

a)

```
SELECT
    get_json_object(user_profile, 'user_id') AS user_id,
    get_json_object(user_profile, 'name')    AS name,
    get_json_object(user_profile, 'age')     AS age,
    get_json_object(user_profile, 'email')   AS email
FROM bronze_table;
```

b)

```
SELECT
    user_profile.user_id          AS user_id,
    user_profile.name             AS name,
    CAST(user_profile.age AS INT) AS age,
    user_profile.email            AS email
FROM bronze_table;
```

c)

```
SELECT
    from_json(
        user_profile,
        'user_id STRING, name STRING, age STRING, email STRING'
    ) AS parsed_profile
FROM bronze_table;
```

d)

```
SELECT
    get_json_object(user_profile, '$.user_id') AS user_id,
    get_json_object(user_profile, '$.name')    AS name,
    CAST(get_json_object(user_profile, '$.age') AS INT) AS age,
    get_json_object(user_profile, '$.email')   AS email
FROM bronze_table;
```

52.

```
deduped_df = df.dropDuplicates()
deduped_df.summary("count", "mean", "stddev").show()
from pyspark.sql.functions import approx_count_distinct
deduped_df.select(approx_count_distinct("user_id")).show()
```

A developer is building a data pipeline that processes records from a bronze table into a silver table. The bronze table, bronze_events, contains duplicate records due to at-least-once delivery guarantees from the upstream ingestion system. The developer writes the following PySpark code to prepare the data:
The developer notices that after running dropDuplicates () without arguments, some rows that differ only in the event_timestamp column remain. The developer wants to de- duplicate based only on user_id and event_type, keeping one row per unique combination of these two columns.
Which code change achieves this deduplication requirement?

a) Replace df.dropDuplicates() with df.groupBy("user_id", "event_type") .count ()

b) Replace df.dropDuplicates() with df.distinct().select ("user_id", "event_type")

c) Replace df.dropDuplicates() with df.dropDuplicates(["user_id", "event_type"])

d) Replace df.dropDuplicates() with df.drop("event_timestamp") .dropDuplicates ()

53. A data engineering team is analyzing customer transactions using a PySpark DataFrame named transactions with columns customer_id, amount, and category. The team needs to compute, for each customer, the total transaction amount, the average transaction amount, and the maximum amount - all in a single operation for efficiency.
Which code fragment should be used to achieve this in PySpark?

a)
```
transactions.groupBy("customer_id")
    .agg(sum("amount"), avg("amount"), max("amount")))
```

b)
```
transactions.groupBy("customer id")
    .select(sum("amount"), avg("amount"), max("amount"))
```

c)
```
transactions.agg(groupBy("customer_id"),
                 sum("amount"),
                 avg("amount"),
                 max("amount"))
```

d) . transactions.groupBy ("customer_id") sum ("amount") -avq ("amount") .max ("amount")

54. A data engineer needs to compute metrics such as: + rolling 7-day sales + percent of total revenue + top-N most profitable categories.
Which two approaches are appropriate for implementing these metrics with PySpark DataFrames? (Choose two.)

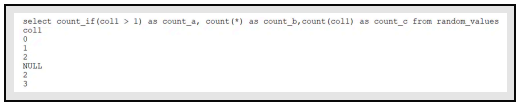
a) Using RDD aggregations for better performance.
b) Using mapPartitions to manually implement rolling windows.
c) Using window functions for rolling and ranking calculations.
d) Collecting the DataFrame to the driver for faster iterative processing.
e) Using joins and groupBy to compute percent-of-total metrics.

55. 5 data engineer wants to create a data entity from a couple of tables. The data entity must be used by other data engineers in other sessions. It also must be saved to a physical location. Which of the following data entities should the data engineer create?

a) Database
b) Function
c) View
d) Temporary view
e) Table

56. Identify a scenario to use an external table. A Data Engineer needs to create a parquet bronze table and wants to ensure that it gets stored in a specific path in an external location.
Which table can be created in this scenario?

a) An external table where the location is pointing to specific path in external location.
b) An external table where the schema has managed location pointing to specific path in external location.
c) A managed table where the catalog has managed location pointing to specific path in external location.
d) A managed table where the location is pointing to specific path in external location.

57. 

```
select count_if(col1 > 1) as count_a, count(*) as count_b,count(col1) as count_c from random_values
col1
0
1
2
NULL
2
3
```

Identify how the count_if function and the count where x is null can be used. Consider a table random_values with below data.
What would be the output of below query?

a) 3,6,5
b) 4,6,5
c) 3,6,6
d) 4,6,6

58. A data engineering team is building a new data transformation notebook. During development, they need fast testing, quick code changes, and easy debugging. Later, the notebook will run nightly as a scheduled job without human intervention. They want to optimize for development speed.
Which compute should they use during development?

a) Use instance pools.
b) Use classic job compute.
c) Use interactive compute.
d) Use SQL warehouses.

59. A data engineer uploads a CSV file using the "Create or modify a table using file upload" option in Databricks. To avoid incorrect schema inference, they disable Automatically detect column types before creating a Unity Catalog-managed table. What is the outcome?

a) All columns are created with the STRING data type.
b) Numeric columns are inferred as numeric types; others are STRING.
c) Previously inferred column types are preserved.
d) Table creation fails because column types are not specified.

60. A data engineer is designing a bronze-to-silver pipeline on the Databricks Data Intelligence Platform. The source system sends daily CSV files, and over time, new optional columns are added. The data engineer wants a storage format and table feature that together ensure the following: » The table prevents writes that do not conform to the defined schema. + The table can evolve its schema to include new optional columns without having to manually recreate the table. » Previous versions of the table can be queried later for debugging and auditing.

Which solution fulfills the requirements?

a) Use a Parquet table with Spark's default schema inference and rely on re-running the job when the schema changes.

b) Use an external table with Auto Loader schema inference for CSV files.

c) Use a Delta table with its native capabilities.

d) Use a Delta table with schema enforcement and recreate the table whenever new columns are added.

61. A data engineer has developed a Python notebook in a Databricks workspace and has configured it to run as a scheduled Job to process daily sales data. How are the storage and execution of this notebook managed within the Databricks architecture?

a) The notebook is stored securely and encrypted in the control plane, and the code executes in the compute plane when the job runs.

b) The notebook is stored securely and encrypted in the compute plane, and the code executes in the control plane when the job runs.

c) The notebook is stored securely and encrypted in Unity Catalog, and the code executes in Delta Lake when the job runs.

d) The notebook is stored unencrypted in the workspace storage bucket, and the code executes on the cluster driver node in the compute plane when the job runs.

62. A data engineer has multiple Unity Catalog-managed Delta tables that require regular maintenance. Currently, the data engineer manually schedules OPTIMIZE and VACUUM jobs for each table, adjusting the frequency based on how often each table is queried and updated. The engineer needs to eliminate this manual maintenance overhead and have the platform automatically determine when and how to run these operations.
Which action should the engineer take to meet the requirements?

a) Enable Liquid Clustering with CLUSTER BY on each table to automatically compact and remove stale data files on a scheduled basis.
b) Set the delta. autoOptimize.autoCompact and delta. autoOptimize.optimizeWrite table properties to trigger automatic VACUUM and file compaction
c) Configure Auto Loader with schema evolution enabled to automatically trigger OPTIMIZE and VACUUM after each data ingestion batch completes.
d) Enable predictive optimization at the account, catalog, or schema level so the platform automatically identifies and runs OPTIMIZE and VACUUM on managed tables.

63. A data engineer must support self-serve BI dashboards for hundreds of business users who run ad hoc, high-concurrency SQL queries throughout the day against governed Unity Catalog tables. The team needs a near-instant start, autoscaling without manual tuning, and the best performance for SQL while minimizing operational overhead.
Which Databricks compute should the data engineer use?

a) SQL Warehouse (Classic) with a fixed cluster size.
b) Jobs compute clusters triggered by dashboards on a schedule.
c) All-purpose classic cluster with manual autoscaling enabled.
d) SQL Warehouse (Serverless) with Photon enabled.

64. A developer has configured Databricks Connect and starts running PySpark code locally. They notice that Data Frame operations are executed on the remote cluster, but actions like . show() return results directly in their local console.
What does this behaviour illustrate about how Databricks Connect is used?

a) It requires manual synchronization of results from the cluster
b) It runs all Spark compute locally and only reads remote files
c) It proxies Spark commands from the local client to the remote cluster
d) It mirrors the entire cluster environment on the developer's machine

65. A data engineer working in Databricks needs to control BI query latency and cost for a sales analytics domain. Silver tables already contain validated, normalized sales and product data. The team wants dashboard-friendly query performance and to align datasets with business concepts (e.g., weekly sales and account_performance) without overloading analysts with complex joins.
Which design approach should a data engineer choose for the Gold layer in a medallion architecture?

a) Move raw, append-on1y source data directly into Gold to avoid transformation costs and enable time travel.

b) Create denormalized, domain-aligned tables and materialized aggregates (€.g., weekly_sales) optimized for reporting and dashboards.

c) Build only lightly cleansed views in Gold and preserve most business logic in Silver to reduce duplication.

d) Keep all data heavily normalized in Gold to minimize storage and rely on BI tools to join data at query time.

66. A data engineer is building a PySpark ingestion pipeline in a local IDE and must execute heavy DataFrame transformations on a remote Databricks cluster for scale while stepping through business logic locally. The workspace uses Unity Catalog and a cluster running Databricks Runtime 15.4 LTS. The engineer also plans to register UDFs.
What should the data engineer do to avoid runtime issues during development with Databricks Connect?

a) Match the local Python minor version to the cluster's Python minor version when using UDF's, and use a Databricks Connect package compatible with the cluster's Runtime version

b) Use any local Python version as long as the PySpark minor version matches, because the cluster's Python version is isolated from the client.

c) Use serverless, because Databricks Connect does not support assigned or shared clusters.

d) Disable Unity Catalog on the workspace because it conflicts with remote execution through Databricks Connect.

67. A data engineer is setting up a new Databricks pipeline that ingests clickstream events from Kafka and daily product catalogs from cloud object storage. To ensure auditability and easy reprocessing, the engineer wants to land all source data. Later stages will handle cleaning, deduplication, and business modeling before the data is used in dashboards.
Which approach aligns to Medallion Architecture principles?

a) Land both sources in Gold with denormalized star schemas to optimize BI while retaining full source fidelity.
b) Land streaming events from Kafka in silver, and the product catalog directly in Gold to minimize layers for batch data.
c) Land both sources in the Bronze layer append-only with minimal validation, then build Silver/Gold downstream for quality and analytics.
d) Land both sources directly into the Silver layer with schema enforcement and deduplication to reduce downstream complexity.

68. A data engineer requires rapid iteration on pipelines while maintaining reliable rollbacks after bad ingests, ensuring audit trails for regulatory compliance, and providing consistent access to a single source of truth for both AI and BI workloads.
Which strategy should the data engineer apply to meet the needs?

a) Delta Lake ACID transactions and time travel, governed by Unity Catalog for consistent access and lineage.
b) DBFS CSV storage with manual file versioning and nightly copies for rollback.
c) Ephemeral in-memory DataFrames for audit trails and BI distribution.
d) Cloud objects storage only, with ad hoc SQL queries for recovery and governance.

69. A data engineer must deliver a trustworthy customer 360 dataset in Databricks for data scientists and BI teams. The engineer plans to join deduplicated customer records with cleaned transaction data, enforce schema and data quality checks, and create a conformed “customer_transactions” view. Later, highly aggregated, domain-specific tables (for weekly spend and executive summaries) will be produced for dashboards.
Where should the engineer build the conformed “customer_transactions” dataset, and where should the aggregated, report-ready tables reside?

a) Build both “customer_transactions” and aggregated, report-ready tables in Silver to keep the model simpler.
b) Build “customer_transactions” in Bronze and put the aggregated, report-ready tables in Silver.
c) Build “customer_transactions” in Gold and put the aggregated, report-ready tables in Silver.
d) Build “customer transactions” in Silver and put the aggregated, report-ready tables in Gold.

70. A data engineer needs to conduct Exploratory Analysis on data residing in a database that is within the company's custom-defined network in the cloud. The data engineer is using SQL for this task.
Which type of SQL Warehouse will enable the data engineer to process large numbers of queries quickly and cost effectively?

a) Classic SQL Warehouse
b) Serverless SQL Warehouse
c) Pro SQL Warehouse
d) All-purpose compute cluster

71. A data engineering team wants to validate a new ingestion pipeline locally while ensuring large aggregations run in serverless compute in their Databricks workspace. They plan to use Databricks Connect and have the option to attach to either a shared cluster or serverless.
Which workspace requirement should be confirmed first to avoid connection failures?

a) Verify that the workspace has Unity Catalog disabled and the Databricks Connect version is less than the serverless Runtime version.
b) Verify that the workspace has Unity Catalog enabled and that the Databricks Connect version supports serverless for the target Runtime release.
c) Verify that only assigned access mode clusters are used because serverless is not supported by Databricks Connect.
d) Verify that the local Spark version equals the serverless Spark version to satisfy Spark Connect parity.

72. A data engineer needs to combine sales data from an on-premises PostgreSQL database with customer data in Azure Synapse for a comprehensive report. The goal is to avoid data duplication and ensure up-to-date information.
How should the data engineer achieve this using Databricks?

a) Export data from both sources to CSV files and upload them to Databricks
b) Use Lakehouse Federation to query both data sources directly
c) Manually synchronize data from both sources into a single database
d) Develop custom ETL pipelines to ingest data into Databricks

73. What is the primary function of the Silver layer in the Databricks medallion architecture?

a) Store historical data solely for auditing purposes
b) Aggregate and enrich data for business analytics
c) Validate, clean, and deduplicate data for further processing
d) Ingest raw data in its original state

74. A Data Engineer is designing Bronze layer in Databricks Medallion Architecture. The raw data is collected from multiple sources (Clickstream in JSON, Transactions in CSV). The task is to design the Bronze layer of the Medallion Architecture to ingest and store this raw data for further processing.
Which operation applies to the Bronze layer?

a) Ingest raw data without transformations, preserving original schemas, and store in Delta format.
b) Clean and standardize raw data by removing null values and enforcing schemas.
c) Apply complex business logic to enrich raw data with customer segmentation labels.
d) Aggregate and transform source data to calculate daily sales performance metrics.

75. Which languages are supported by Serverless compute clusters?
(Choose two.)

a) SQL
b) Python
c) R
d) Scala
e) Java

76. A data engineer has written a function in a Databricks Notebook to calculate the population of bacteria in a given medium. def calculate population of bacteria (intital_population, exponential factor) return future population = intitial population ** expontential factor Analysts use this function in the notebook and sometimes provide input arguments of the wrong data type, which can cause errors during execution.
Which Databricks feature will help the data engineer quickly identify if an incorrect data type has been provided as input?

a) The Spark User interface has a Debug tab that contains the variables used in this session.
b) The Databricks debugger enables breakpoints that will raise an error if the wrong data type is submitted.
c) The Databricks debugger enables the use of a variable explorer to see at a glance the value of the variables.
d) The Data Engineer should add print statements to determine the value of the variable.

77. An organization has implemented a data pipeline in Databricks and needs to ensure it can scale automatically based on varying workloads without manual cluster management. The goal is to meet the company's Service Level Agreements (SLAs), which require high availability and minimal downtime, while Databricks automatically handles resource allocation and optimization.
Which approach fulfills these requirements?

a) Deploy Job Clusters with fixed configurations, dedicated to specific tasks, without automatic scaling.

b) Use Spot Instances to allocate resources dynamically while minimizing costs, with potential interruptions.

c) Use Interactive Clusters in Databricks, adjusting cluster sizes manually based on workload demands.

d) . Use Serverless compute in Databricks to automatically scale and provision resources with minimal manual intervention.

78. A data engineer needs to process SQL queries on a large dataset with fluctuating workloads. The workload requires automatic scaling based on the volume of queries, without the need to manage or provision infrastructure. The solution should be cost-efficient and charge only for the compute resources used during query execution.
Which compute option should the data engineer use?

a) Databricks SQL Analytics

b) Databricks Runtime for ML

c) Databricks Jobs

d) Serverless SQL Warehouse

79. A data engineer is developing a small proof of concept in a notebook. When running the entire notebook, the Cluster usage spikes. The data engineer wants to keep the development requirements and get real-time results.
Which Cluster meets these requirements?

a) All Purpose Cluster with autoscaling

b) Job Cluster with Photon enabled and autoscaling

c) Job Cluster with autoscaling enabled

d) All-Purpose Cluster with a large fixed memory size

80. A data engineer wants to create an external table in Databricks that references data stored in an Azure Data Lake Storage (ADLS) location. The goal is to enable Databricks to access and query this external data without moving it into the Databricks-managed storage.
Which step should the data engineer take to successfully create the external table?

a) Use the CREATE MANAGED TABLE statement and specify the LOCATION clause with the path to the external data.
b) CREATE UNMANAGED TABLE statement without specifying a LOCATION clause.
c) Use the CREATE TABLE statement and specify the LOCATION clause with the path to the external data.
d) CREATE EXTERNAL TABLE statement without specifying a LOCATION clause.

81. A data engineer is debugging a Python notebook in Databricks that processes a dataset using PySpark. The notebook fails with an error during a DataFrame transformation. The engineer wants to inspect the state of variables, such as the input DataFrame and intermediate results, to identify where the error occurs.
Which tool should the engineer use to debug the notebook and inspect the values of variables like DataFrames?

a) Use the Databricks CLI to download and analyze driver logs for detailed error messages
b) Use the Python Notebook Interactive Debugger to set breakpoints and inspect variable values in real-time
c) Use the Ganglia UI to monitor cluster resource usage and identify hardware issues
d) Use the Spark UI to analyze the execution plan and identify stages where the job failed

82. What is the maximum output supported by a job cluster to ensure a notebook does not fail?

a) 25MBs
b) 10MBs
c) 30MBs
d) 15MBs

83. A data engineer is working on a personal laptop and needs to perform complex transformations on data stored in a Delta Lake on cloud storage. The engineer decides to use Databricks Connect to interact with Databricks clusters and work in their local IDE.
How does Databricks Connect enable the engineer to develop, test, and debug code seamlessly on their personal laptop while interacting with Databricks clusters?

a) By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using a specific IDE that is required by Databricks

b) By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code only through Databricks' own web interface

c) By allowing direct execution of Spark jobs from the local machine without needing a network connection

d) By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using their preferred IDE

84. Which compute option should be chosen in a scenario where small scale ad-hoc Python scripts need to be run at high frequency and should wind down quickly after these queries have finished running?

a) All-purpose Cluster

b) Job Cluster

c) Serverless Compute

d) SQL Warehouse

85. A data engineer is attempting to write Python and SQL in the same command cell and is running into an error. The engineer believed it was possible to use a Python variable in a SQL select statement.
Why does the command fail?

a) Databricks supports language interoperability in the same cell but only between Scala and SQL.

b) Databricks supports multiple languages but only one per notebook.

c) Databricks supports one language per cell.

d) Databricks supports language interoperability but only if a special character is used.

86. An organization needs an optimized storage layer that supports ACID transactions and schema enforcement.
Which technology should the organization use?

a) Delta Lake
b) Unity Catalog
c) Cloud File Storage
d) Data lake

87. How can Git operations be performed outside of Databricks Repos?

a) Commit
b) Pull
c) Merge
d) Clone

88. Which two components function in the DB platform architecture's control plane? (Choose two.)

a) Virtual Machines
b) Compute Orchestration
c) Serverless Compute
d) Compute
e) Unity Catalog

89. Which file format is used for storing Delta Lake Table?

a) CSV
b) Parquet
c) JSON
d) Delta

90. A data engineer is running code in a Databricks Repo that is cloned from a central Git repository. A colleague of the data engineer informs them that changes have been made and synced to the central Git repository. The data engineer now needs to sync their Databricks Repo to get the changes from the central Git repository.
Which Git operation does the data engineer need to run to accomplish this task?

a) Clone
b) Pull
c) Merge
d) Push

Answer Keys

1. b) Run databricks bundle deploy --target prod so the deployment uses targets .prod and overrides dataset_catalog to catalog_prod
2. a) Compare cluster metrics in Lakeflow Jobs run history to spot resource spikes causing delays.
3. b) Define bundle variables and reference them from bundle resources, then override variable values per environment using bundle targets.
4. b) Add the dashboard task as dependent on all three parallel tasks using fan-in control flow.
5. b) The File Arrival trigger starts the job run, while Auto Loader uses its internal checkpoint to independently identify and process only the new files that have arrived since the last successful commit.
6. c) Run databricks bundle generate for each existing resource, use --bind to link the generated configuration to the existing workspace resources, and commit the bundle project to Git for CI/CD deployment.
7. b) Query the Lakeflow Spark Declarative Pipelines event log to retrieve expectation metrics and failure details for the pipeline's most recent update.
8. d) Spark UI opened from the job run's compute, using the Jobs and Stages tabs to inspect task metrics.
9. d) Repair run
10. a) Use Jobs REST API repairRun to re-run failed tasks; the repair uses the current job and task settings and appears in the run history.
11. b) A databricks.yml YAML file that contains the bundle's name, targets, includes resources, and variables
12. b) Place exactly one databricks.yml at the repository root; it is the main configuration file and may reference additional configuration files via the include mapping.
13. a) Run the job on a serverless compute for workflows configuration,
14. b) Repair the run
15. c) Scheduled Workflows can reduce resource consumption and

ensuring Unity Catalog is enabled and regional support is available.

expense since the cluster runs only long enough to execute the pipeline.

16. a) The pipeline can have different notebook sources in SQL & Python.

17. c) Jobs Cluster

18. c) All datasets will be updated at set intervals until the pipeline is shut down. The compute resources will be deployed for the update and terminated when the pipeline is stopped.

19. b) They could wrap the queries using PySpark and use Python's control flow system to determine when to run the final query.

20. b) Load corrections as new files and apply updates to the target table

21. a) A Lakeflow Connect managed Salesforce connector that writes into Unity Catalog Delta tables

22. b) Restart the streaming job, so it picks up the new managed table location

23. b) The new userAgent field is added as a new nullable column in the Delta table, populated for new data.

24. b) Use Structured Streaming for Kafka and Lakeflow Connect managed connector with incremental mode for Snowflake.

25. a) Use a Lakeflow Connect managed connector for the SaaS app with built-in CDC, schema evolution, OAuth, and serverless compute with retries, then process downstream with Lakeflow Spark Declarative Pipelines.

26. a) Use file notification mode because it scales efficiently for both large numbers of existing files and continuous new arrivals by using cloud storage events rather than repeatedly listing directories.

27. c) Use Lakeflow Connect managed connectors for Salesforce and SQL Server, and Auto Loader for ADLS files, all within Lakeflow Spark Declarative Pipelines.

28. d)

29. c) WHEN NOT MATCHED BY SOURCE THEN DELETE

30. a)

31. d)

32. a) trigger (availableNow=True)

33. d)

34. a) Checkpointing and Write-ahead Logs

35. d) trigger (processingTime="5 seconds")

36. d) trigger (processingTime="5 seconds")

37. a) WHERE customer_id <=> :cid

38. b) orders df and customers_df inner join

39. c) Materialized view

40. b) The stage likely suffers from data skew.

41. d) spark.driver.memory is too low to hold the collected result set

42. d) broadcast country *lookup*df when performing the join

43. c) spark.sql.autoBroadcastJoinThreshold

44. b) left anti-join from customers to orders

45. a) df. dropna(subset= ['customer_email', 'customer_phone'])

46. a) df.dropDuplicates(['customer_id', 'email'])

47. b) Use spark.read.format ("jdbc") and write with saveAsTable ("catalog.schema. table")

48. a) A materialized view that precomputes aggregations from Silver tables on a schedule and is queried by BI tools for faster, consistent analytics.

49. a)

50. b) For Pipeline A, increase the spark.driver.memory; for Pipeline B, increase the spark.sql.shuffle.partitions or executor memory.

51. d)

52. c) Replace df.dropDuplicates() with df.dropDuplicates(["user_id", "event_type"])

53. a)

54. c) Using window functions for rolling and ranking calculations., e) Using joins and groupBy to compute percent-of-total metrics.

55. e) Table

56. a) An external table where the location is pointing to specific path in external location.

57. a) 3,6,5

58. c) Use interactive compute.

59. a) All columns are created with the STRING data type.

60. c) Use a Delta table with its native capabilities.

61. a) The notebook is stored securely and encrypted in the control plane, and the code executes in the compute plane when the job runs.

62. d) Enable predictive optimization at the account, catalog, or schema level so the platform automatically identifies and runs OPTIMIZE and VACUUM on managed tables.

63. d) SQL Warehouse (Serverless) with Photon enabled.

64. c) It proxies Spark commands from the local client to the remote cluster

65. b) Create denormalized, domain-aligned tables and materialized aggregates (€.g., weekly_sales) optimized for reporting and dashboards.

66. a) Match the local Python minor version to the cluster's Python minor version when using UDF's, and use a Databricks Connect package compatible with the cluster's Runtime version

67. c) Land both sources in the Bronze layer append-only with minimal validation, then build Silver/Gold downstream for quality and analytics.

68. a) Delta Lake ACID transactions and time travel, governed by Unity Catalog for consistent access and lineage.

69. d) Build "customer transactions" in Silver and put the aggregated, report-ready tables in Gold.

70. b) Serverless SQL Warehouse

71. b) Verify that the workspace has Unity Catalog enabled and that the Databricks Connect version supports serverless for

72. b) Use Lakehouse Federation to query both data sources directly

the target Runtime release.

73. c) Validate, clean, and deduplicate data for further processing

74. a) Ingest raw data without transformations, preserving original schemas, and store in Delta format.

75. a) SQL , b) Python

76. b) The Databricks debugger enables breakpoints that will raise an error if the wrong data type is submitted.

77. d) . Use Serverless compute in Databricks to automatically scale and provision resources with minimal manual intervention.

78. d) Serverless SQL Warehouse

79. a) All Purpose Cluster with autoscaling

80. c) Use the CREATE TABLE statement and specify the LOCATION clause with the path to the external data.

81. b) Use the Python Notebook Interactive Debugger to set breakpoints and inspect variable values in real-time

82. b) 10MBs

83. d) By providing a local environment that mimics the Databricks runtime, enabling the engineer to develop, test, and debug code using their preferred IDE

84. c) Serverless Compute

85. c) Databricks supports one language per cell.

86. a) Delta Lake

87. d) Clone

88. b) Compute , Unity Orchestration e) Catalog

89. b) Parquet

90. b) Pull